PROYECTO ETL: SEGUNDA ENTREGA

JHEFFREY URBANO GARCES
ALEXANDER MOSQUERA FAJARDO

ETL
M.Sc. DANIEL FELIPE ROMERO BERNAL

UNIVERSIDAD AUTÓNOMA DE OCCIDENTE
FACULTAD DE INGENIERÍA Y CIENCIAS BÁSICAS
INGENIERÍA DE DATOS E INTELIGENCIA ARTIFICIAL
SANTIAGO DE CALI
2026

1. Introducción y Evolución del Proyecto **3**
2. Arquitectura Tecnológica e Infraestructura en la Nube **3**
3. Integración de Múltiples Fuentes de Datos Heterogéneas **3**
4. Diseño del Flujo Orquestado en Apache Airflow (DAG) **4**
5. Resultados y Valor Analítico **4**
6. Conclusiones **4**
7. Aparte Técnico: Del Entorno Local (Jupyter Notebook) al Despliegue en Producción (GCP) **5**
A. Modularización del Código (De Celdas a Scripts Reutilizables) 5
B. Infraestructura y Orquestación en Google Cloud Platform (GCP) 5
8. Conclusiones, Hallazgos Analíticos e Impacto del Proyecto ETL **5**

GitHub Repositorio

https://github.com/JheffreyUrbano/etl_entrega_2

1. Introducción y Evolución del Proyecto

En la primera etapa de nuestro proyecto, construimos un modelo relacional en estrella basado en los registros históricos de presuntos suicidios en Colombia (2015-2024), aplicando procesos de limpieza y estructuración directamente en un entorno estático.

Para esta Segunda Entrega, el reto consistió en evolucionar hacia una arquitectura de nivel de producción. Dejamos atrás los scripts aislados para implementar un pipeline automatizado, orquestado, resiliente y validado por contratos de datos, manteniendo nuestro firme compromiso social alineado con el Objetivo de Desarrollo Sostenible (ODS) 3: Salud y Bienestar (Meta 3.4).

2. Arquitectura Tecnológica e Infraestructura en la Nube

Para garantizar la reproducibilidad y el aislamiento de los servicios, desplegamos toda la infraestructura en una Máquina Virtual de Google Cloud Platform (GCP) conectada a un entorno robusto de contenedores Docker:

Orquestación con Apache Airflow (2.10.2): Configurado mediante un archivo docker-compose.yml personalizado para coordinar la ejecución diaria (@daily) de todas las tareas del flujo.
Nuevo Contenedor de Base de Datos (etl_entrega_2): Un contenedor independiente de PostgreSQL corriendo en el puerto 5433 que aloja nuestra base de datos consolidada suicidios_colombia, permitiendo consultas analíticas limpias sin interferir con otros procesos.
Almacenamiento Cloud (ws2-etl-bucket): Utilizado para almacenar de forma segura los respaldos estructurados y los archivos CSV complementarios.

3. Integración de Múltiples Fuentes de Datos Heterogéneas

Para enriquecer el análisis de salud pública y pasar de meros conteos absolutos a indicadores proporcionales, el pipeline automatiza la ingesta de cinco fuentes de datos diferentes:

Base de Datos Histórica (PostgreSQL / Docker): Extrae los registros originales de Medicina Legal estructurados en la primera entrega.
Proyecciones de Población (DANE - Archivo CSV en Bucket GCS): Ingiere las proyecciones demográficas municipales para permitir el cálculo de tasas por cada 100.000 habitantes, neutralizando el sesgo de densidad poblacional de las grandes ciudades.
Indicador Macroeconómico (API del Banco Mundial): Consume de forma automatizada y resiliente (usando adaptadores HTTP con reintentos y backoff exponencial) la serie histórica de la tasa de desempleo anual en Colombia (2015-2024) para medir impactos económicos post-pandemia.
Calendario de Festivos (API de Nager.Date): Realiza peticiones iterativas anuales para inyectar los días festivos y puentes oficiales de Colombia, enriqueciendo la dimensión temporal para detectar ventanas temporales de vulnerabilidad.

Geografía Departamental (Web Scraping de Wikipedia): Extrae mediante BeautifulSoup y tablas HTML normalizadas las características físicas, superficie y altitud de los municipios del Valle del Cauca.

4. Diseño del Flujo Orquestado en Apache Airflow (DAG)

El flujo de trabajo automatizado fue programado bajo un grafo acíclico dirigido (etl_suicidios_valle_v2) compuesto por cuatro etapas secuenciales y paralelas:

- **Fase 1 - Extracción Paralela (Extract):** Cinco tareas independientes descargan y guardan los datos en la capa de staging local (/opt/airflow/data) aplicando controles contra fallos de red.
- **Fase 2 - Transformación y Cruce (Transform & Merge):** Un script centralizado en Pandas limpia nulos, homologa formatos de texto, estandariza fechas y ejecuta operaciones de cruce (merge) utilizando códigos DANE, años y nombres normalizados de municipios.
- **Fase 3 - Validación de Calidad con Great Expectations (Quality Check):** Antes de escribir en la base de datos final, se ejecuta un contrato de datos estricto que evalúa reglas críticas:
    - Conteo de filas mayor a cero (evita datasets vacíos).
    - Ausencia de nulos en llaves primarias, años y campos obligatorios.
    - Validación de rangos lógicos para el año (2015-2025) y la tasa de desempleo (0% a 50%).
    - Dominios cerrados en variables categóricas esenciales como el sexo.

    (**Nota:** Si alguna regla falla, el pipeline se detiene automáticamente para evitar contaminar el Data Warehouse).
- **Fase 4 - Carga Final (Load):** Al superar las validaciones, se cargan las nuevas tablas dimensionales (dim_economia_worldbank, dim_festivos_nager, dim_municipios_scraping) y la tabla de hechos enriquecida (fact_suicidios_enriquecida_v2) directamente en PostgreSQL utilizando SQLAlchemy.

5. Resultados y Valor Analítico

Gracias a la consolidación de este pipeline, logramos responder a los interrogantes planteados en los objetivos de la entrega:

- **Incidencia Real Per Cápita:** Al cruzar las cifras con las proyecciones del DANE, comprobamos que municipios con menor población absoluta en el Valle del Cauca experimentan tasas proporcionales de riesgo que requieren atención focalizada, descentralizando el análisis de la ciudad de Cali.
- **Correlación Socioeconómica y Estacional:** La automatización permitió vincular las curvas de desempleo y los fines de semana largos (festivos), ofreciendo a los analistas de salud pública una base sólida para entender los detonantes contextuales de las crisis de salud mental.

6. Conclusiones proceso

La implementación de esta Segunda Entrega demostró el verdadero valor de la ingeniería de datos moderna. Pasamos de procesar archivos estáticos a diseñar un ecosistema totalmente automatizado, tolerante a fallos, respaldado en la nube y protegido por estrictos controles de calidad de datos, cumpliendo a cabalidad con los estándares académicos y profesionales exigidos.

7. Aparte Técnico: Del Entorno Local (Jupyter Notebook) al Despliegue en Producción (GCP)

Uno de los mayores retos de ingeniería en esta segunda entrega fue trasladar la lógica que inicialmente prototipamos de forma local en cuadernos de Jupyter (.ipynb) hacia un entorno de producción real, automatizado y accesible de manera remota. Este salto tecnológico requirió resolver tres pilares fundamentales:

A. Modularización del Código (De Celdas a Scripts Reutilizables)
En la fase inicial (primera entrega), el código residía en bloques secuenciales dentro de celdas de Jupyter, donde las variables compartidas en memoria facilitaban la ejecución manual. Para llevarlo a un flujo automatizado en Apache Airflow, refactorizamos todo ese código monolítico en módulos de Python estructurados y desacoplados:

- **Scripts de Extracción (extract_sources.py):** Encapsularon las peticiones a APIs con reintentos, el scraping de Wikipedia y la lectura de buckets en funciones independientes.
- **Scripts de Transformación (transform_merge.py) y Validación (validate_ge.py):** Aislaron la lógica de limpieza en Pandas y el contrato de Great Expectations.
- **Scripts de Carga (load_dw.py):** Automatizaron la conexión por SQLAlchemy hacia PostgreSQL.

B. Infraestructura y Orquestación en Google Cloud Platform (GCP)
Para garantizar que el pipeline operara de forma continua sin depender de una computadora local, realizamos el despliegue en una Máquina Virtual (Compute Engine) de Google Cloud:

- **Aislamiento con Docker y Docker Compose:** Creamos un ecosistema contenerizado que aísla los servicios de Airflow (Webserver, Scheduler, Base de datos analítica etl_entrega_2) mediante volúmenes compartidos (/opt/airflow/data). Esto evita conflictos de dependencias del sistema operativo y asegura que el entorno de desarrollo y el de producción sean exactamente iguales.
- **Gestión de Permisos e Identidad (IAM):** Configuramos cuentas de servicio y llaves de acceso en GCP para permitir que tanto los contenedores como los colaboradores del proyecto (jurbano) interactuaran de forma segura con los buckets de almacenamiento (ws2-etl-bucket), resolviendo restricciones de red y políticas de seguridad corporativa o de nube.

8. Conclusiones, Hallazgos Analíticos e Impacto del Proyecto ETL

A lo largo del desarrollo de esta Segunda Entrega, el proyecto evolucionó desde un ejercicio estático de limpieza de datos hasta la consolidación de un ecosistema en producción automatizado, orquestado y validado en la nube. Más allá del cumplimiento técnico de la arquitectura, el análisis integral de las cinco fuentes de datos nos permitió extraer conclusiones profundas que demuestran el verdadero valor de la ingeniería de datos aplicada a la salud pública y al ODS 3 (Salud y Bienestar):

- Superación del Sesgo Demográfico y Hallazgos Geográficos (Zoom Regional) El peligro de los números absolutos: En la primera etapa del análisis, observamos que Santiago de Cali concentraba más de la mitad de los registros totales de presuntos suicidios en el departamento del Valle del Cauca. Sin embargo, este hallazgo inicial presentaba un sesgo evidente provocado por la alta densidad poblacional de la capital frente a los demás municipios.

El poder de la normalización por Habitante: Gracias a la integración automatizada del archivo CSV con las Proyecciones de Población del DANE en nuestro pipeline, transformamos los conteos brutos en tasas estandarizadas por cada 100.000 habitantes. Al visualizar estos datos normalizados, descubrimos que municipios intermedios e industriales (como Yumbo, Palmira, Jamundí o Tuluá) exhiben tasas proporcionales de incidencia sumamente críticas que antes quedaban ocultas bajo la sombra estadística de Cali. Este hallazgo demuestra a los expertos en salud pública que las estrategias de prevención no deben concentrarse de manera centralizada, sino descentralizada, focalizando recursos y líneas de atención en los territorios con mayor riesgo relativo.

- Correlación Macroeconómica y Ventanas Temporales Impacto del ciclo económico: Mediante el consumo resiliente de la API del Banco Mundial, logramos inyectar la serie histórica de la tasa de desempleo nacional (2015–2024). Al cruzar esta variable macroeconómica con el comportamiento anual de los casos, el modelo visualiza de forma clara cómo el repunte sostenido de reportes observado entre 2021 y 2023 coincide con los efectos colaterales de la desaceleración económica y la presión laboral post-pandemia. Esto valida técnicamente la hipótesis planteada en nuestro Análisis Exploratorio de Datos (EDA) inicial, conectando los factores financieros con las crisis de salud mental.

Estacionalidad y calendario: La incorporación de la API de Nager.Date permitió automatizar la inyección de los días festivos oficiales del país. Aunque los casos se distribuyen a lo largo de los días ordinarios, el etiquetado temporal automatizado dota al Data Warehouse de una dimensión estacional lista para que los epidemiólogos analicen si los puentes festivos y periodos vacacionales representan ventanas de mayor vulnerabilidad social.

- Rigor Técnico, Gobernanza y Calidad del Dato Arquitectura tolerante a fallos: El diseño del flujo en Apache Airflow demostró cómo aislar componentes mediante contenedores Docker (separando el motor de metadatos de Airflow del Data Warehouse analítico en el puerto 5433) garantiza una ejecución programada (@daily) libre de interferencias con otros proyectos del servidor.

Gobernanza activa con Cuarentena y Eventos: La implementación de Great Expectations redefinió nuestro rol como ingenieros de datos. Aprendimos que un pipeline profesional no descarta datos a ciegas ni permite que registros corruptos dañen los tableros gerenciales. Al programar el enrutamiento automático de anomalías hacia la tabla cuarentena_suicidios y registrar cada auditoría en audit_quality_events, logramos proteger la integridad del Data Warehouse (fact_suicidios_enriquecida_v2) al tiempo que conservamos una trazabilidad forense completa de cualquier error institucional o de formato.

- Conclusión General del Equipo Como estudiantes de ingeniería de datos, este proyecto consolidó nuestra comprensión sobre el ciclo de vida completo de la información. Comprendimos que la construcción de un Data Warehouse no se trata únicamente de escribir código de programación, sino de diseñar soluciones éticas, reproducibles y robustas en la nube. Un sistema de datos automatizado, validado y transparente es la herramienta fundamental para transformar registros crudos y aislados en conocimiento útil que oriente decisiones reales orientadas a salvar vidas y mejorar el bienestar social.